

Cuadro comparativo – Diseño UML propio vs diseño sugerido por IA

Prompt utilizado para solicitar revisión a la IA

Actúa como Arquitecto de Software Principal y Consultor Senior de Java. Analiza el diagrama UML de un sistema hotelero y propone mejoras aplicando principios de Programación Orientada a Objetos, herencia, abstracción, encapsulamiento, composición, agregación y persistencia de datos.

Resumen de la recomendación realizada por IA

La IA propuso separar el sistema en capas, implementar clases abstractas, utilizar herencia, incorporar DAOs, enums y mejorar la modularidad.

Tabla comparativa

Elemento analizado	Diseño con IA	Diseño sin IA	Decisión final del grupo	Justificación
Estructura general del sistema	Arquitectura dividida en modelo, DAO y GUI	Diseño centrado únicamente en clases principales del negocio	Se adoptó una estructura parcialmente por capas	Facilita organización y mantenimiento del sistema
Clase Cliente	Clase abstracta Cliente con herencia	Cliente manejado de forma más simple	Se mantuvo Cliente abstracto	Permite reutilizar atributos y métodos comunes
Tipos de cliente	PersonaNatural y Empresa con subtipos	Menor nivel de especialización	Se conservaron los subtipos principales	Mejora flexibilidad y manejo de descuentos
Clase Habitación	Clase abstracta con herencia	Habitaciones modeladas de forma básica	Se adoptó herencia en habitaciones	Permite extender tipos de habitación fácilmente
Tipos de habitación	Sencilla, Doble y Matrimonial	Menor diferenciación de atributos	Se conservaron tipos específicos	Representa mejor la realidad del hotel
Gestión de reservas	Clase Reserva centralizada y relacionada	Reserva menos integrada	Se mantuvo modelo centralizado	Facilita control del flujo del sistema
Servicios adicionales	Inclusión de servicios consumidos	Manejo limitado de servicios	Se agregaron servicios consumidos	Permite calcular costos adicionales
Facturación	Clase Factura integrada	Facturación menos	Se añadió facturación	Mejora control financiero

		detallada	formal	
Persistencia de datos	Uso de clases DAO	No existía capa DAO	Se adoptó parcialmente DAO	Favorece separación entre lógica y almacenamiento
Estados del sistema	Uso de enums	Estados manuales	Se incorporaron enums	Reduce errores y mejora claridad
Interfaces y modularidad	Uso de interfaces/capas DAO y separación modular	Diseño más directo y menos desacoplado	Se adoptó parcialmente la modularidad propuesta	Mejora mantenibilidad y escalabilidad

Explicación breve de las decisiones de diseño

El diseño final integró elementos del diseño realizado por el grupo y las recomendaciones obtenidas mediante IA, aplicando principios de Programación Orientada a Objetos como herencia, abstracción, encapsulamiento y composición.

Explicación adicional sobre interfaces y modularidad

El diseño sugerido por IA incorporó una arquitectura más modular, utilizando separación por capas y estructuras similares a interfaces mediante DAOs y componentes especializados. Esto permitió reducir el acoplamiento entre clases y mejorar la escalabilidad del sistema.